LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 8

IMBALANCED DATASET



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Mahasiswa memahami konsep imbalanced dataset serta permasalahan yang muncul akibat ketidakseimbangan kelas dalam model machine learning
2. Mahasiswa mampu menerapkan teknik balancing data seperti SMOTE dan ADASYN untuk menyeimbangkan distribusi kelas pada dataset
3. Mahasiswa memahami konsep dan metode feature selection (filter, wrapper, embedded) untuk memilih fitur yang paling relevan

B Dasar Teori

B.1 Imbalanced Class

Imbalanced class atau ketidakseimbangan kelas merupakan kondisi pada klasifikasi di mana distribusi data antar kelas target tidak seimbang secara signifikan, ditandai dengan salah satu kelas yang mendominasi (*majority class*) sementara kelas lainnya memiliki jumlah sampel yang jauh lebih sedikit (*minority class*) [1]. Kondisi ini menjadi permasalahan serius dalam pembelajaran mesin karena mayoritas algoritma klasifikasi dirancang untuk memaksimalkan akurasi keseluruhan, sehingga model cenderung bias ke arah kelas mayoritas dan mengabaikan kelas minoritas yang justru sering kali merupakan kelas krusial, seperti pada kasus deteksi kecurangan transaksi maupun diagnosis medis [1].

B.2 Confusion Matrix

Confusion matrix adalah tabel kontingensi yang digunakan untuk mengukur dan mengevaluasi kinerja suatu model klasifikasi dengan memetakan perbandingan antara hasil prediksi model dengan label kelas sebenarnya, yang terdiri dari elemen *True Positive* (TP), *True Negative* (TN), *False Positive* (FP), dan *False*

Negative (FN) [2]. Berdasarkan nilai-nilai dalam matriks ini, dapat dihitung berbagai metrik evaluasi krusial seperti *precision*, *recall*, *F1-score*, dan *specificity*, yang memberikan penilaian performa yang jauh lebih komprehensif dan objektif daripada sekadar metrik akurasi dasar, terutama saat menangani data dengan distribusi kelas yang timpang [2].

B.3 Feature Selection

Feature selection atau seleksi fitur adalah proses pemilihan subset fitur yang paling relevan dan informatif dari sekumpulan atribut pada data asli untuk digunakan dalam proses pelatihan model pembelajaran mesin [3]. Penerapan seleksi fitur bertujuan untuk mereduksi dimensionalitas data, mengeliminasi atribut yang bersifat redundan atau mengandung *noise*, mempercepat waktu pelatihan komputasi, serta meningkatkan interpretabilitas dan performa generalisasi model klasifikasi dengan meminimalkan risiko terjadinya *overfitting* [3].

B.4 Filter Method

Filter method merupakan pendekatan seleksi fitur yang mengevaluasi dan menyaring fitur berdasarkan karakteristik statistik intrinsik dari data tanpa melibatkan algoritma model pembelajaran mesin dalam proses pemilihannya [4]. Metode ini memanfaatkan perhitungan statistik seperti korelasi Pearson, *Chi-square*, atau *Information Gain* untuk memberikan bobot atau peringkat (*ranking*) pada setiap atribut, sehingga memiliki keunggulan efisiensi komputasi yang sangat cepat dan skalabilitas tinggi untuk data berdimensi masif, meskipun memiliki keterbatasan karena tidak mempertimbangkan interaksi antarfitur maupun keterkaitannya dengan model klasifikasi yang digunakan [4].

B.5 Wrapper Method

Wrapper method adalah pendekatan seleksi fitur yang mengikutsertakan model pembelajaran mesin tertentu sebagai fungsi evaluasi untuk mengukur keefektifan dari suatu subset kombinasi fitur [5]. Melalui strategi pencarian sistematis seperti *forward selection*, *backward elimination*, atau *recursive feature elimination* (RFE), metode ini secara iteratif melatih dan mengevaluasi model untuk menemukan kombinasi fitur yang menghasilkan performa prediksi paling optimal untuk algoritma yang dipilih, kendati membutuhkan beban komputasi yang jauh lebih berat dan memiliki kerentanan *overfitting* pada dataset berskala kecil [5].

B.6 Embedded Method

Embedded method merupakan teknik seleksi fitur yang proses pemilihannya terintegrasi langsung di dalam proses pelatihan algoritma pembelajaran mesin itu sendiri [6]. Pendekatan ini menggabungkan efisiensi komputasi dari *filter method* serta kemampuan interaksi fitur yang presisi dari *wrapper method*, salah satunya diwujudkan melalui teknik regularisasi seperti Lasso (L_1 *regularization*) yang secara matematis menyusutkan koefisien fitur yang kurang signifikan menjadi tepat nol, maupun berbasis pohon seperti *Random Forest* yang menghitung nilai tingkat kepentingan fitur (*feature importance*) saat pohon dibangun [6].

B.7 Synthetic Minority Over-sampling Technique (SMOTE)

Synthetic Minority Over-sampling Technique (SMOTE) adalah salah satu teknik *data-level* berbasis *oversampling* yang bekerja dengan membangkitkan sampel data sintetis baru untuk kelas minoritas, bukan sekadar menggandakan data yang telah ada [7]. Algoritma SMOTE beroperasi dengan cara mengidentifikasi tetangga terdekat (*k-nearest neighbors*) dari setiap sampel kelas minoritas, kemudian membuat titik data buatan baru di sepanjang garis penghubung antara sampel tersebut den-

gan tetangganya, sehingga dapat memperluas batas keputusan (*decision boundary*) model terhadap kelas minoritas tanpa menimbulkan permasalahan *overfitting* yang parah [7].

B.8 Adaptive Synthetic (ADASYN)

Adaptive Synthetic (ADASYN) adalah algoritma pengembangan dari metode SMOTE yang menggunakan pendekatan adaptif dalam menghasilkan sampel sintetis untuk kelas minoritas berdasarkan tingkat kesulitan pembelajarannya [8]. ADASYN secara otomatis menghitung distribusi bobot untuk setiap sampel minoritas berdasarkan proporsi sampel kelas mayoritas yang berada di sekitarnya, sehingga sampel yang berada di area perbatasan dan lebih sulit dipelajari oleh model akan menghasilkan jumlah data sintetis yang lebih banyak dibandingkan sampel yang mudah dipelajari, yang secara efektif menggeser fokus pembelajaran model ke arah area yang paling rentan terhadap kesalahan klasifikasi [8].

C Hasil dan Pembahasan

C.1 Import library

Pada tahap awal proses analisis data, dilakukan proses import library yang dibutuhkan untuk melakukan analisis data. Library yang digunakan adalah sebagai berikut:

```
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import RandomOverSampler
from scipy.stats import norm
from imblearn.over_sampling import ADASYN
from imblearn.over_sampling import SMOTE

import seaborn as sns
import matplotlib.pyplot as plt
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Gambar 1: Import library

C.2 Import dan Load Dataset Information

Mahasiswa mengunduh dataset terlebih dahulu pada link yang disediakan `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset?resource=download`. Panggil file csv yang telah diunduh tersebut menggunakan pandas, lalu tampilkan 5 data pertama.

```
# read file
df = pd.read_csv('healthcare-dataset-stroke-data.csv')

# baris data dari atas
df.head()
```

✓ 0.0s

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 2: Import dan load dataset information

C.3 EDA (Exploratory Data Analysis)

Visualisasi distribusi variabel target (*stroke*) dilakukan menggunakan *pie chart* untuk mengidentifikasi keberadaan ketidakseimbangan kelas (*imbalanced class*) pada

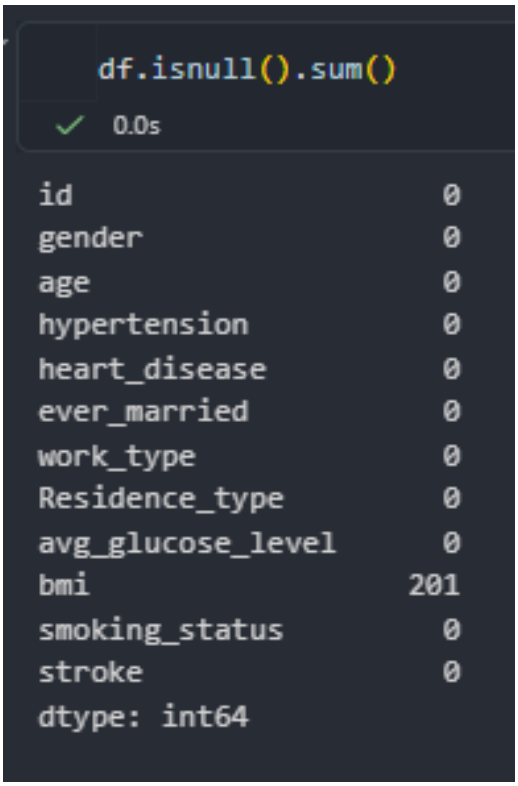
dataset.



Gambar 3: Visualisasi distribusi variabel target

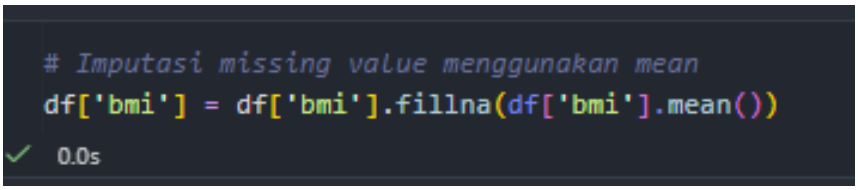
Berdasarkan Gambar 3, dataset menunjukkan ketidakseimbangan kelas yang sangat ekstrem (*highly imbalanced data*), dengan kelas 0 (tidak stroke) mendominasi sebesar 95,13% dan kelas 1 (stroke) hanya sebesar 4,87%. Ketimpangan proporsi yang tajam ini dapat menyebabkan model klasifikasi bias ke arah kelas mayoritas, sehingga penerapan teknik penyeimbangan data seperti *undersampling*, *oversampling*, SMOTE, maupun ADASYN sangat diperlukan sebelum tahap pemodelan.

C.4 Data Cleaning



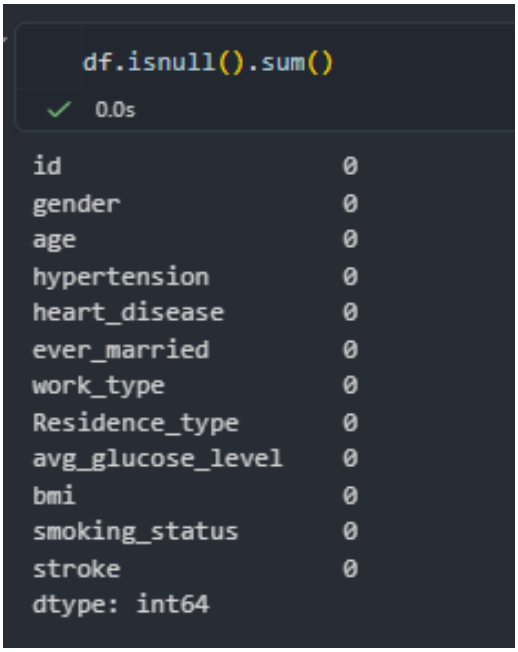
Gambar 4: Cek missing value

Berdasarkan gambar 4, terdapat 201 baris yang kosong pada kolom bmi. Oleh karena itu diperlukan imputation untuk mengisi semua data kosong tersebut dengan nilai rata-rata dari kolom bmi.



Gambar 5: Mengisi value kosong dengan nilai rerata

Setelah melakukan imputation, lihat kembali dengan mengecek missing value. Dapat dilihat pada gambar 6, jika semua kolom tidak ada missing value.



Gambar 6: Tidak ada missing value

C.5 Random Undersampling

Penerapan *Random Undersampling* dilakukan dengan mereduksi jumlah data pada kelas mayoritas (kelas 0) secara acak agar seimbang dengan jumlah data kelas minoritas (kelas 1).

```python
from imblearn.under_sampling import RandomUnderSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

under_sampling = RandomUnderSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

le = LabelEncoder()
df_y = le.fit_transform(df_y)

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender' ] = le.fit_transform(df_X['gender'])
df_X['ever_married' ] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type' ])
df_X['Residence_type' ] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status' ] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25, random_state=42)

X_resampled, y_resampled = under_sampling.fit_resample(X_train, y_train)

print('sebelum undersampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah undersampling')
print(pd.DataFrame(y_resampled).value_counts())
```

✓ 0.0s

```
sebelum undersampling
0
0    3663
1     169
Name: count, dtype: int64
setelah undersampling
0
0    169
1    169
Name: count, dtype: int64
```

Gambar 7: Penerapan kode Random Undersampling

Berdasarkan Gambar 7, jumlah sampel kelas 0 yang semula berjumlah 3.663 dipangkas secara acak menjadi 169 sampel, sehingga kedua kelas memiliki jumlah data yang seimbang yaitu masing-masing 169 data.

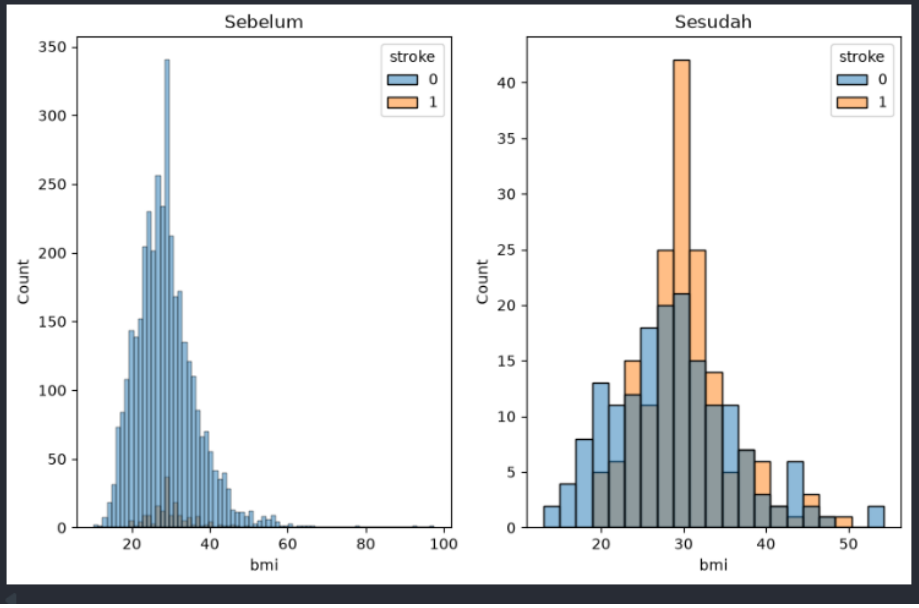
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks(rotation='horizontal')
plt.show()
```

0.7s



Gambar 8: Histogram fitur bmi sebelum dan sesudah Random Undersampling

Berdasarkan Gambar 8, distribusi frekuensi fitur *bmi* setelah *undersampling* mengalami penurunan skala jumlah data secara drastis, dengan proporsi kemunculan kelas 0 dan kelas 1 yang kini setara di seluruh rentang nilai *bmi*.

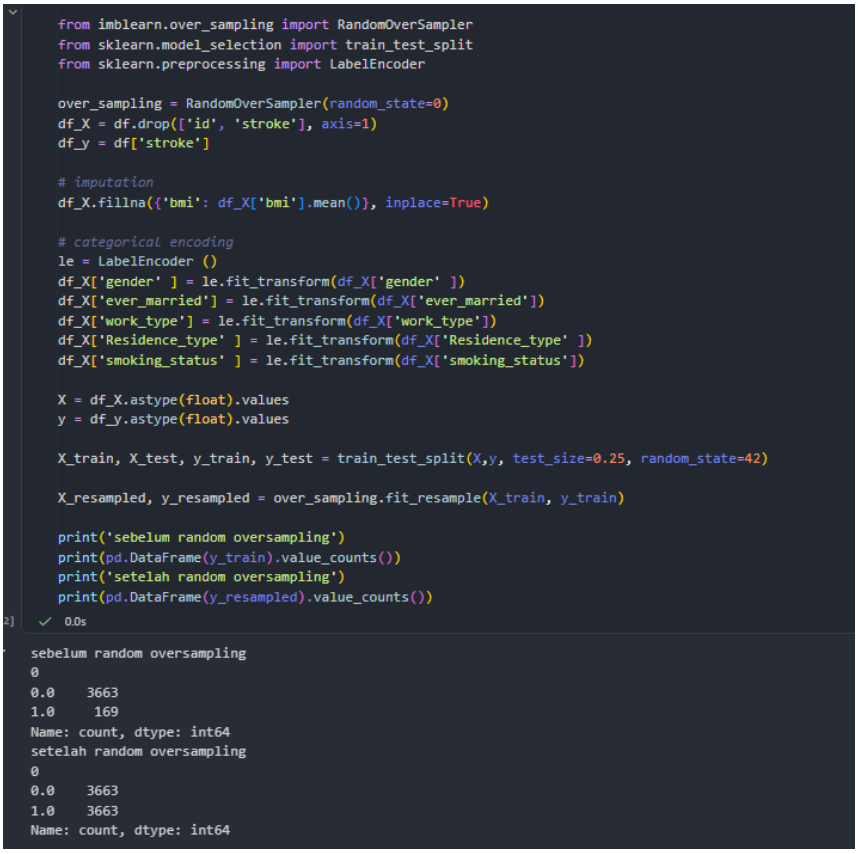


Gambar 9: Scatterplot persebaran data sebelum dan sesudah Random Undersampling

Berdasarkan Gambar 9, kepadatan titik kelas 0 (biru) berkurang secara signifikan sehingga persebarannya menjadi berimbang dengan titik kelas 1 (oranye), meskipun terdapat banyak informasi dari kelas mayoritas yang terbuang.

C.6 Random Oversampling

Metode *Random Oversampling* dilakukan dengan menduplikasi sampel kelas minoritas (kelas 1) secara acak hingga jumlahnya menyamai kelas mayoritas (kelas 0).



```
from imblearn.over_sampling import RandomOverSampler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

over_sampling = RandomOverSampler(random_state=0)
df_X = df.drop(['id', 'stroke'], axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder ()
df_X['gender' ] = le.fit_transform(df_X['gender' ])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type' ] = le.fit_transform(df_X['Residence_type' ])
df_X['smoking_status' ] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.25, random_state=42)

X_resampled, y_resampled = over_sampling.fit_resample(X_train, y_train)

print('sebelum random oversampling')
print(pd.DataFrame(y_train).value_counts())
print('setelah random oversampling')
print(pd.DataFrame(y_resampled).value_counts())
```

```
0.0s
sebelum random oversampling
0
0.0    3663
1.0     169
Name: count, dtype: int64
setelah random oversampling
0
0.0    3663
1.0    3663
Name: count, dtype: int64
```

Gambar 10: Penerapan kode Random Oversampling

Berdasarkan Gambar 10, jumlah sampel kelas 1 berhasil ditingkatkan secara signifikan dari 169 menjadi 3.663 data, menghasilkan distribusi data latih yang seimbang sempurna dengan rasio 1:1.

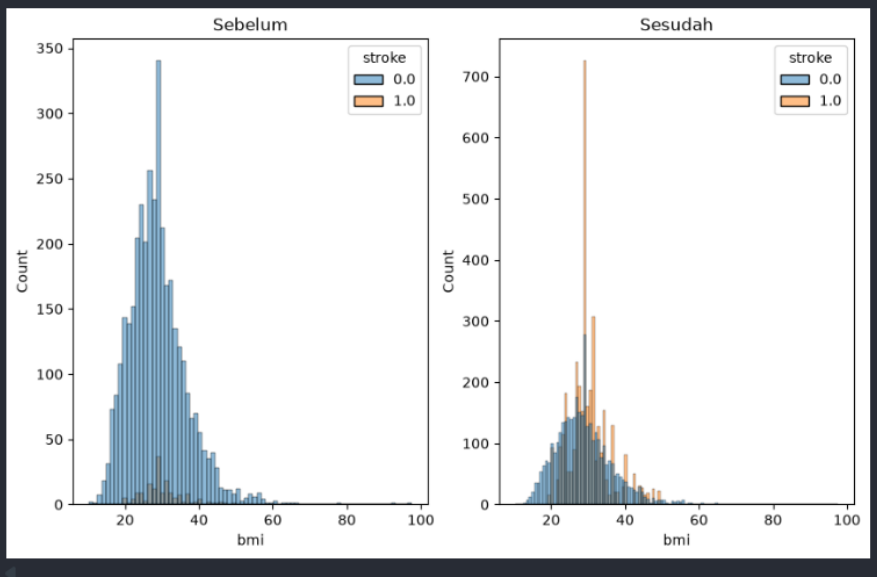
```python
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```

✓ 1.3s



Gambar 11: Histogram fitur bmi sebelum dan sesudah Random Oversampling

Berdasarkan Gambar 11, terlihat peningkatan tajam frekuensi kelas 1 pada fitur *bmi* hingga menyamai tinggi kelas 0, dengan lonjakan runcing pada nilai-nilai tertentu akibat duplikasi langsung sampel yang sama secara berulang.

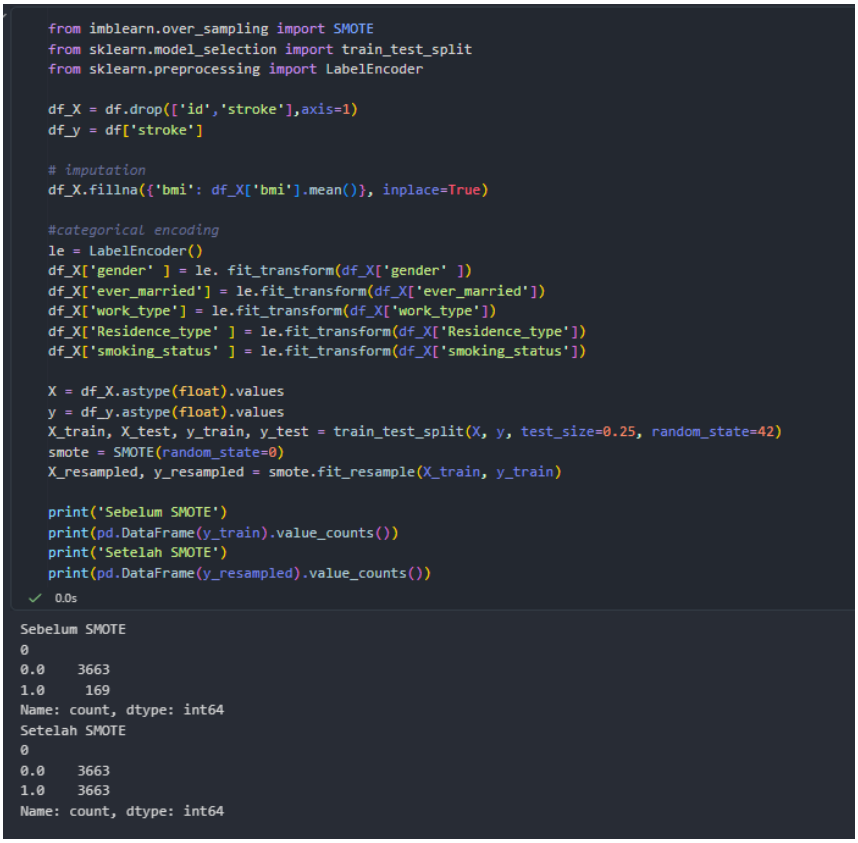


Gambar 12: Scatterplot persebaran data sebelum dan sesudah Random Oversampling

Berdasarkan Gambar 12, titik-titik kelas 1 (oranye) tampak semakin tebal dan bertumpuk pada posisi koordinat data yang sama dengan data aslinya, karena tidak ada titik data baru yang dihasilkan melainkan hanya penggandaan sampel yang sudah ada.

C.7 SMOTE

Teknik SMOTE (*Synthetic Minority Over-sampling Technique*) menyeimbangkan data dengan membangkitkan sampel sintetis baru di antara sampel minoritas terdekat menggunakan pendekatan interpolasi k-NN.



```
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id','stroke'],axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

#categorical encoding
le = LabelEncoder()
df_X['gender' ] = le. fit_transform(df_X['gender' ])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type' ] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status' ] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
smote = SMOTE(random_state=0)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

print('Sebelum SMOTE')
print(pd.DataFrame(y_train).value_counts())
print('Setelah SMOTE')
print(pd.DataFrame(y_resampled).value_counts())
```

✓ 0.0s

```
Sebelum SMOTE
0
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah SMOTE
0
0.0    3663
1.0    3663
Name: count, dtype: int64
```

Gambar 13: Penerapan kode SMOTE

Berdasarkan Gambar 13, proses penyeimbangan dengan SMOTE berhasil meningkatkan jumlah kelas minoritas menjadi 3.663 data sintetis, menghasilkan keseimbangan penuh dengan kelas 0 tanpa menduplikasi data identik.

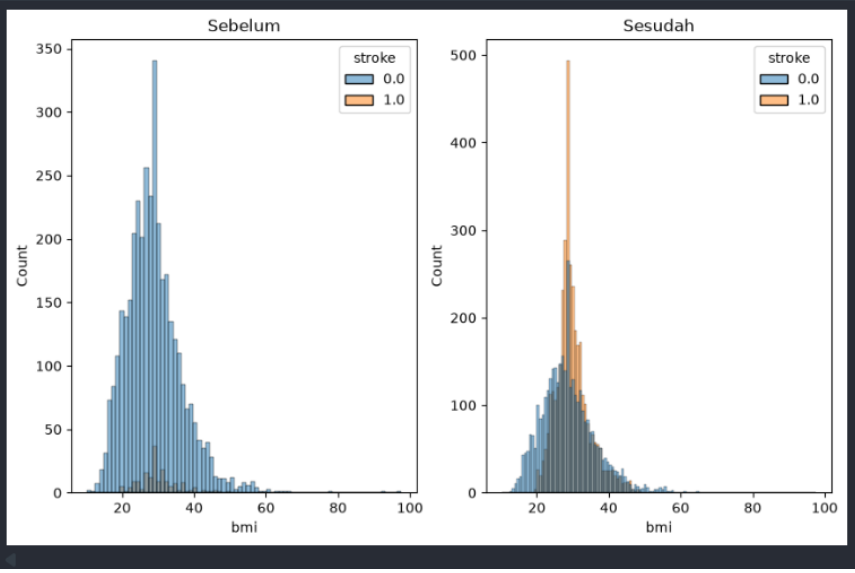
```
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot(data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot(data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```

1.6s



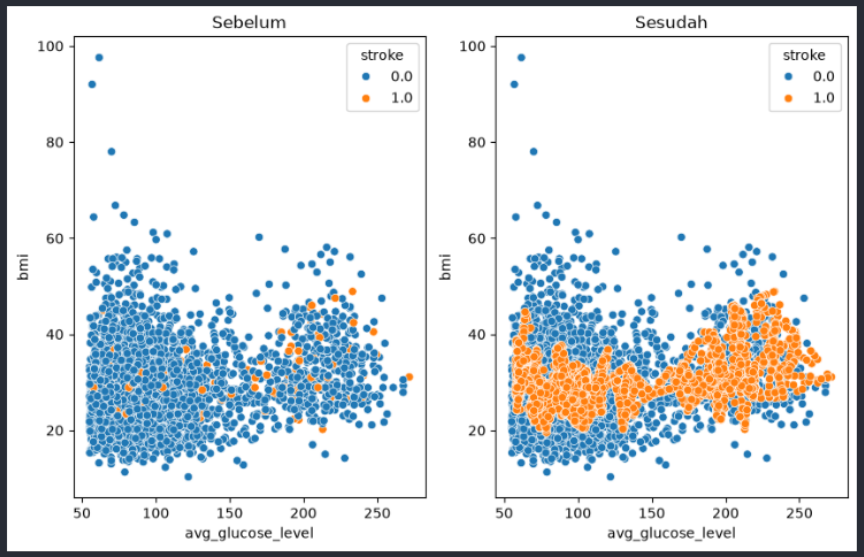
Gambar 14: Histogram fitur bmi sebelum dan sesudah SMOTE

Berdasarkan Gambar 14, kurva sebaran kelas 1 pada fitur *bmi* meningkat secara merata dan memiliki bentuk distribusi yang menyerupai sebaran kelas mayoritas, menunjukkan variasi data buatan yang lebih representatif.

```
import seaborn as sns

f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sebelum, hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.scatterplot(x='avg_glucose_level', y='bmi', data=df_sesudah, hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```

✓ 0.7s



Gambar 15: Scatterplot persebaran data sebelum dan sesudah SMOTE

Berdasarkan Gambar 15, titik-titik sintetis oranye baru terbentuk mengisi ruang-ruang di antara sampel minoritas asli, memperluas area keputusan kelas 1 secara kontinu dan menghindari penumpukan titik data yang identik.

C.8 ADASYN

Metode ADASYN (*Adaptive Synthetic*) merupakan variasi adaptif dari SMOTE yang menghasilkan lebih banyak sampel sintetis pada area kelas minoritas yang sulit dipelajari atau berdekatan dengan kelas mayoritas.



```
from imblearn.over_sampling import ADASYN
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

df_X = df.drop(['id','stroke'],axis=1)
df_y = df['stroke']

# imputation
df_X.fillna({'bmi': df_X['bmi'].mean()}, inplace=True)

# categorical encoding
le = LabelEncoder()
df_X['gender' ] = le.fit_transform(df_X['gender' ])
df_X['ever_married'] = le.fit_transform(df_X['ever_married'])
df_X['work_type'] = le.fit_transform(df_X['work_type'])
df_X['Residence_type' ] = le.fit_transform(df_X['Residence_type'])
df_X['smoking_status' ] = le.fit_transform(df_X['smoking_status'])

X = df_X.astype(float).values
y = df_y.astype(float).values
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
adasyn = ADASYN(random_state=0)
X_resampled, y_resampled = adasyn. fit_resample(X_train, y_train)

print('Sebelum ADASYN')
print(pd.DataFrame(y_train).value_counts())
print('Setelah ADASYIN')
print(pd.DataFrame(y_resampled).value_counts())
```

✓ 0.0s

```
Sebelum ADASYN
0
0.0    3663
1.0     169
Name: count, dtype: int64
Setelah ADASYIN
0
1.0    3676
0.0    3663
Name: count, dtype: int64
```

Gambar 16: Penerapan kode ADASYN

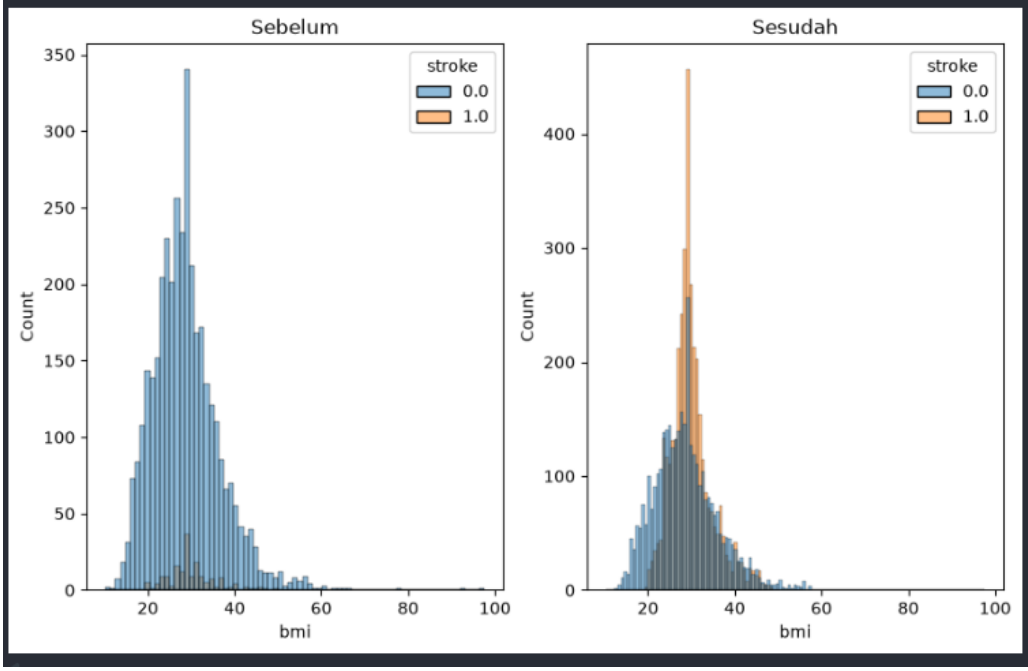
Berdasarkan Gambar 16, ADASYN membangkitkan 3.676 sampel sintetis untuk kelas 1, sehingga distribusinya seimbang dengan kelas 0 (3.663 data) dengan penyesuaian bobot adaptif.

```python
from scipy.stats import norm
import seaborn as sns
import matplotlib.pyplot as plt

dfx_ori= pd.DataFrame(X_train, columns=df_X.columns)
dfy_ori = pd.DataFrame(y_train, columns=['stroke'])
df_sebelum = pd.concat([dfx_ori, dfy_ori], axis=1)

dfx_res = pd.DataFrame(X_resampled, columns=df_X.columns)
dfy_res = pd.DataFrame(y_resampled, columns=['stroke'])
df_sesudah = pd.concat([dfx_res, dfy_res], axis=1)
f, axes = plt.subplots(1,2, figsize=(10, 6))
sns.histplot (data=df_sebelum, x='bmi',hue='stroke', ax=axes[0]).set(title='Sebelum')
sns.histplot (data=df_sesudah, x='bmi',hue='stroke', ax=axes[1]).set(title='Sesudah')
plt.xticks()
plt.show()
```

0.9s



Gambar 17: Histogram fitur bmi sebelum dan sesudah ADASYN

Berdasarkan Gambar 17, frekuensi kelas 1 pada fitur *bmi* meningkat seimbang dengan kelas 0, dengan konsentrasi data sintetis yang lebih terfokus pada rentang nilai dengan tingkat ambiguitas klasifikasi tinggi.



Gambar 18: Scatterplot persebaran data sebelum dan sesudah ADASYN

Berdasarkan Gambar 18, sebaran data sintetis kelas 1 (oranye) tampak sangat padat dan terfokus pada batas keputusan (*decision boundary*) antara kelas 0 dan kelas 1, membantu model dalam memisahkan kedua kelas yang tumpang tindih secara lebih efektif.

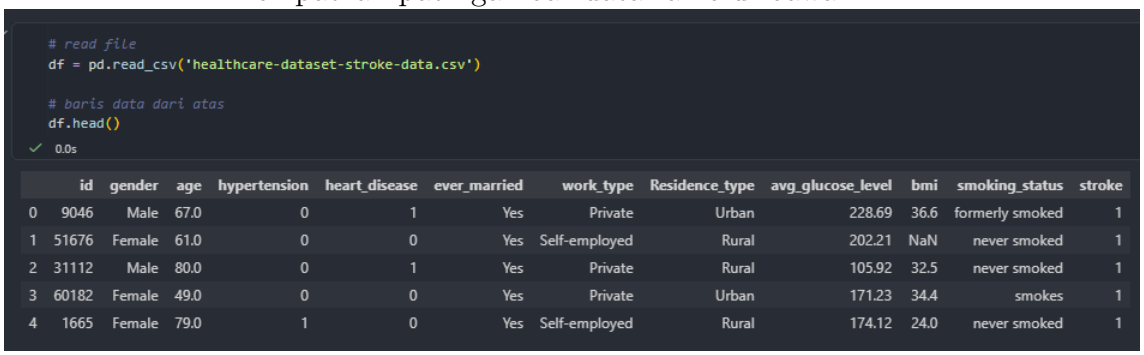
D Tugas dan Analisis

D.1 Tugas 1

Dataset *Stroke Prediction Dataset* diunduh melalui Kaggle dengan link `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset?resource=download` dan disimpan pada repositori kerja untuk kemudian dimuat serta dita-

mpilkan ke dalam bentuk *dataframe* menggunakan pustaka Pandas.

Tempatkan path gambar dataframe di bawah ini:



```
# read file
df = pd.read_csv('healthcare-dataset-stroke-data.csv')

# baris data dari atas
df.head()
```

✓ 0.0s

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 19: Tampilan dataset dalam bentuk dataframe

Tujuan Penggunaan Dataset:

Tujuan utama penggunaan dataset ini adalah untuk membangun model pembelajaran mesin yang mampu mengklasifikasi dan memprediksi risiko kemungkinan seorang pasien terkena stroke berdasarkan kondisi kesehatan fisik, gaya hidup, serta faktor demografis.

Fitur-fitur Dataset:

Dataset terdiri dari 11 atribut fitur dan 1 variabel target:

1. `id`: Nomor identitas unik setiap pasien.
2. `gender`: Jenis kelamin pasien (*Male*, *Female*, *Other*).
3. `age`: Usia pasien.
4. `hypertension`: Riwayat hipertensi (0 = tidak memiliki, 1 = memiliki).
5. `heart_disease`: Riwayat penyakit jantung (0 = tidak memiliki, 1 = memiliki).
6. `ever_married`: Status pernikahan (*No* atau *Yes*).
7. `work_type`: Kategori pekerjaan (*children*, *Govt_job*, *Never_worked*, *Private*, *Self-employed*).
8. `Residence_type`: Kategori tempat tinggal (*Rural* atau *Urban*).

9. `avg_glucose_level`: Rata-rata kadar glukosa dalam darah pasien.

10. `bmi`: Indeks Massa Tubuh (*Body Mass Index*).

11. `smoking_status`: Status merokok pasien (*formerly smoked*, *never smoked*, *smokes*, *Unknown*).

12. `stroke`: Variabel target penentu riwayat stroke (1 = terkena stroke, 0 = tidak terkena stroke).

D.2 Tugas 2

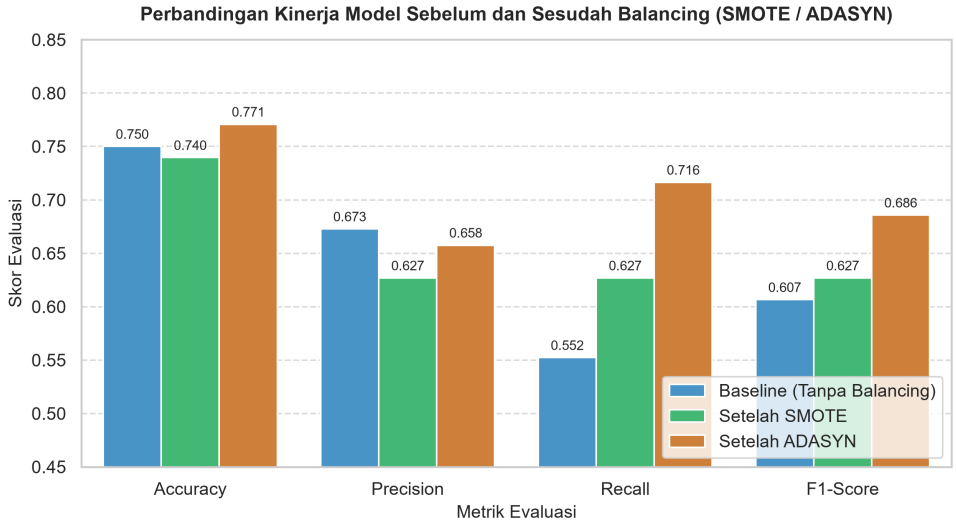
Pada Tugas 2, dilakukan eksperimen penanganan *imbalanced dataset* menggunakan *Pima Indians Diabetes Dataset*. Dataset ini memiliki distribusi target yang tidak seimbang (500 non-diabetes vs 268 diabetes). Dataset dibagi menjadi data latih (75%) dan data uji (25%) secara terstratifikasi. Teknik *balancing* (SMOTE dan ADASYN) hanya diterapkan pada data latih guna mencegah *data leakage*, kemudian model *Random Forest Classifier* dievaluasi pada data uji yang sama.

```
# --- 1. Model Baseline (Sebelum Balancing) ---
rf_base = RandomForestClassifier(random_state=42).fit(X_train, y_train)
y_pred_base = rf_base.predict(X_test)
-> Baseline: Accuracy=0.7500, Precision=0.6727, Recall=0.5522, F1=0.6066

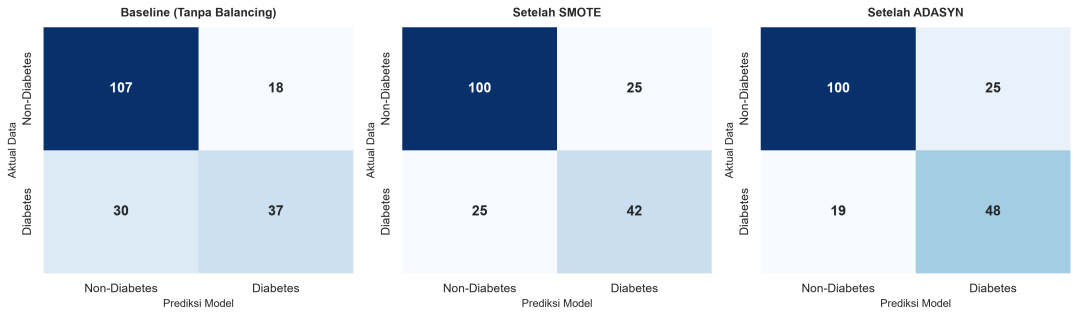
# --- 2. Penerapan SMOTE & Evaluasi Model ---
X_train_sm, y_train_sm = SMOTE(random_state=42).fit_resample(X_train, y_train)
rf_smote = RandomForestClassifier(random_state=42).fit(X_train_sm, y_train_sm)
y_pred_sm = rf_smote.predict(X_test)
-> SMOTE   : Accuracy=0.7396, Precision=0.6269, Recall=0.6269, F1=0.6269  (+7.47% Recall)

# --- 3. Penerapan ADASYN & Evaluasi Model ---
X_train_ada, y_train_ada = ADASYN(random_state=42).fit_resample(X_train, y_train)
rf_adasyn = RandomForestClassifier(random_state=42).fit(X_train_ada, y_train_ada)
y_pred_ada = rf_adasyn.predict(X_test)
-> ADASYN  : Accuracy=0.7708, Precision=0.6575, Recall=0.7164, F1=0.6857  (+16.42% Recall)
```

Gambar 20: Implementasi kode pemodelan dan balancing data



Gambar 21: Perbandingan metrik performa model sebelum dan sesudah data balancing



Gambar 22: Perbandingan confusion matrix (Baseline, SMOTE, dan ADASYN)

Hasil evaluasi performa model sebelum dan sesudah penerapan *data balancing* pada data latih disajikan pada Tabel 1.

Table 1: Perbandingan performa model sebelum dan sesudah data balancing

Skenario Model	**Accuracy**	**Precision**	**Recall**	**F1-Score**
Baseline (Sebelum Balancing)	0.7500	0.6727	0.5522	0.6066
Setelah SMOTE	0.7396	0.6269	0.6269	0.6269
Setelah ADASYN	**0.7708**	0.6575	**0.7164**	**0.6857**

Analisis Singkat Hasil Tugas 2:

1. **Peningkatan Metrik *Recall***: Pada model dasar (*baseline*) tanpa *balancing*, nilai *recall* kelas diabetes hanya sebesar 55,22%. Setelah penerapan SMOTE, nilai *recall* meningkat menjadi 62,69% (+7,47%). Peningkatan tertinggi dicapai oleh metode ADASYN dengan lonjakan *recall* mencapai 71,64% (+16,42%).

2. **Penurunan Kasus *False Negative***: Berdasarkan Gambar 22, jumlah pasien diabetes yang gagal terdeteksi (*False Negative*) menurun drastis dari 30 pasien (Baseline) menjadi 25 pasien (SMOTE), dan tersisa hanya 19 pasien (ADASYN).

3. **Relevansi Medis**: Dalam klasifikasi medis, metrik *recall* memiliki prioritas tertinggi agar dapat meminimalkan kasus pasien diabetes yang tidak terdeteksi secara dini.

E Kesimpulan

Berdasarkan serangkaian praktikum dan eksperimen yang telah dilakukan, dapat disimpulkan bahwa ketidakseimbangan kelas (*imbalanced class*) merupakan tantangan kritis dalam pembelajaran mesin karena menyebabkan model klasifikasi cenderung bias terhadap kelas mayoritas serta menghasilkan performa prediksi yang buruk terhadap kelas minoritas. Penerapan teknik penyeimbangan data berbasis sintesis sampel, yaitu *Synthetic Minority Over-sampling Technique* (SMOTE) dan *Adaptive Synthetic* (ADASYN), terbukti jauh lebih efektif dan representatif dibandingkan teknik konvensional seperti *random undersampling* yang rentan membuang informasi penting maupun *random oversampling* yang berisiko memicu *overfitting* akibat duplikasi data yang identik.

Selain itu, evaluasi komparatif pada dataset medis membuktikan bahwa penerapan SMOTE dan ADASYN pada data latih secara nyata mampu meningkatkan sensitivitas atau metrik *recall* model klasifikasi secara signifikan, yaitu meningkat dari 55,22% pada model *baseline* menjadi 62,69% dengan SMOTE dan mencapai performa tertinggi sebesar 71,64% dengan ADASYN. Peningkatan tersebut berdampak langsung pada reduksi kesalahan *False Negative* secara drastis, yang dalam konteks diagnosis medis merupakan indikator paling krusial guna meminimalkan risiko pasien berpenyakit yang tidak terdeteksi sehingga penanganan medis dapat diberikan sedini mungkin.

F Daftar Pustaka

[1] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[2] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[3] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003.

[4] Y. Saeys, I. Inza, and P. Larrañaga, "A review of feature selection techniques in bioinformatics," *Bioinformatics*, vol. 23, no. 19, pp. 2507–2517, 2007.

[5] R. Kohavi and G. H. John, "Wrappers for feature subset selection," *Artificial Intelligence*, vol. 97, no. 1-2, pp. 273–324, 1997.

[6] R. Tibshirani, "Regression shrinkage and selection via the lasso," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 58, no. 1, pp. 267–288, 1996.

[7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[8] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 2008, pp. 1322–1328.